# Dokumentacja techniczna modułu szablony

17 października 2025

## Spis treści


**1 Wprowadzenie** **2**

**2 Interfejs szablonów — `ITemplate<T>`** **2**
2.1 Opis . . . . . 2

**3 Bazowy szablon — `TemplateBase<T>`** **2**
3.1 Opis . . . . . 3

**4 Dane przeciwnika — `EnemyData`** **3**
4.1 Opis . . . . . 3

**5 Logika przeciwnika — `EnemyBase`** **3**
5.1 Opis . . . . . 4

**6 Dane wieżyczki — `TowerData`** **4**
6.1 Opis . . . . . 4

**7 Logika wieżyczki — `TowerBase`** **4**
7.1 Opis . . . . . 5

**8 Komponenty statusów — `StatusComponent`** **5**
8.1 Opis . . . . . 5
8.2 Przykład: Paralyzed . . . . . 5

**9 Integracja z Unity** **5**

# 1 Wprowadzenie

Moduł `Templates` definiuje zestaw klas i struktur danych do tworzenia elementów gry Tower-Defense w sposób modularny i niezależny od cyklu życia Unity. Celem jest:

- oddzielenie **logiki** (np. ruch, atak, śmierć) od **danych konfiguracyjnych**,
- umożliwienie testowania kodu w czystym C# bez zależności od Unity API,
- wykorzystanie `ScriptableObject` wyłącznie jako kontenerów danych,
- stworzenie solidnych klas bazowych pod przyszłą integrację z komponentami Unity.

# 2 Interfejs szablonów — `ITemplate<T>`

```
public interface ITemplate<T>
{
    string Name { get; }
    void OnCreation(T instance);
    void OnUpdate(T instance, float deltaTime);
    void OnDelete(T instance);
}
```

## 2.1 Opis

- Określa minimalny zestaw funkcji niezbędnych do definiowania zachowań obiektów gry.
- `T` reprezentuje dowolny typ instancji (np. `Enemy`, `Tower`).
- Pozwala oddzielić logikę cyklu życia od danych.

# 3 Bazowy szablon — `TemplateBase<T>`

```
public abstract class TemplateBase<T> : ScriptableObject, ITemplate
   <T>
{
    [Header("Informacje ogolne")]
    public string Name;
    public Sprite Icon;

    public virtual void OnCreation(T instance) { }
    public virtual void OnUpdate(T instance, float deltaTime) { }
    public virtual void OnDelete(T instance) { }
}
```

### 3.1 Opis

- Bazowy szablon danych typu `ScriptableObject`.
- Zawiera wyłącznie pola konfiguracyjne (np. nazwa, ikona).
- Metody cyklu życia są deklaracjami, które mogą być implementowane w klasach logiki.
- Dzięki temu można łączyć dane (`Template`) z logiką (`System / Controller`).

## 4 Dane przeciwnika — `EnemyData`

```
[CreateAssetMenu(menuName = "TDCore/EnemyData")]
public class EnemyData : ScriptableObject
{
    [Header("Podstawowe dane")]
    public string EnemyName;
    public float MaxHealth;
    public float Speed;

    [Header("Wyglad")]
    public Sprite EnemySprite;
}
```

### 4.1 Opis

- Klasa przechowująca konfigurację przeciwnika: nazwę, punkty życia, prędkość i sprite.
- Nie zawiera logiki — może być używana przez systemy zarządzające przeciwnikami.
- Może być przypisana do prefabów lub instancji w grze.

## 5 Logika przeciwnika — `EnemyBase`

```
public abstract class EnemyBase
{
    public EnemyData Data { get; private set; }
    public float CurrentHealth { get; protected set; }
    public Vector2 Position { get; set; }

    public EnemyBase(EnemyData data)
    { ... }

    public virtual void OnCreation() { }
    public virtual void OnUpdate(float deltaTime) { }
    public virtual void OnDeath() { }

    public void ApplyStatus(StatusComponent status) { }
}
```

### 5.1 Opis

- Klasa logiczna przeciwnika, niezależna od Unity.
- Odpowiada za zachowanie i stan jednostki.
- `Data` — referencja do danych konfiguracyjnych (`EnemyData`).
- `OnCreation()`, `OnUpdate()`, `OnDeath()` — metody cyklu życia jednostki.

## 6 Dane wieżyczki — TowerData

```
[CreateAssetMenu(menuName = "TDCore/TowerData")]
public class TowerData : ScriptableObject
{
    [Header("Podstawowe dane")]
    public string TowerName;
    public float AttackRange;
    public float AttackCooldown;

    [Header("Wyglad")]
    public Sprite TowerSprite;
}
```

### 6.1 Opis

- Dane konfiguracyjne wieżyczki.
- Używane przez systemy odpowiedzialne za generowanie lub sterowanie wieżyczkami.

## 7 Logika wieżyczki — TowerBase

```
public abstract class TowerBase
{
    public TowerData Data { get; private set; }
    public float CooldownTimer { get; protected set; }

    public TowerBase(TowerData data)
    { ... }

    public virtual void OnCreation() { }
    public virtual void OnUpdate(float deltaTime) { }
    public virtual void OnAttack(EnemyBase target) { }
}
```

## 7.1 Opis

- Klasa logiczna dla wieżyczek — definiuje mechanikę ataku.
- `TowerData` zawiera dane o zasięgu, szybkości ataku i wyglądzie.
- `OnUpdate()` i `OnAttack()` mogą być nadpisywane w klasach pochodnych.

# 8 Komponenty statusów — `StatusComponent`

```
public abstract class StatusComponent : ScriptableObject
{
    public string StatusName;
    public Sprite StatusIcon;

    public abstract void Apply(EnemyBase enemy);
}
```

## 8.1 Opis

- Definiuje efekty modyfikujące właściwości przeciwnika.
- `StatusIcon` służy do wizualizacji efektu.
- Może być zaimplementowany jako efekt spowolnienia, zatrzymania, trucizny itp.

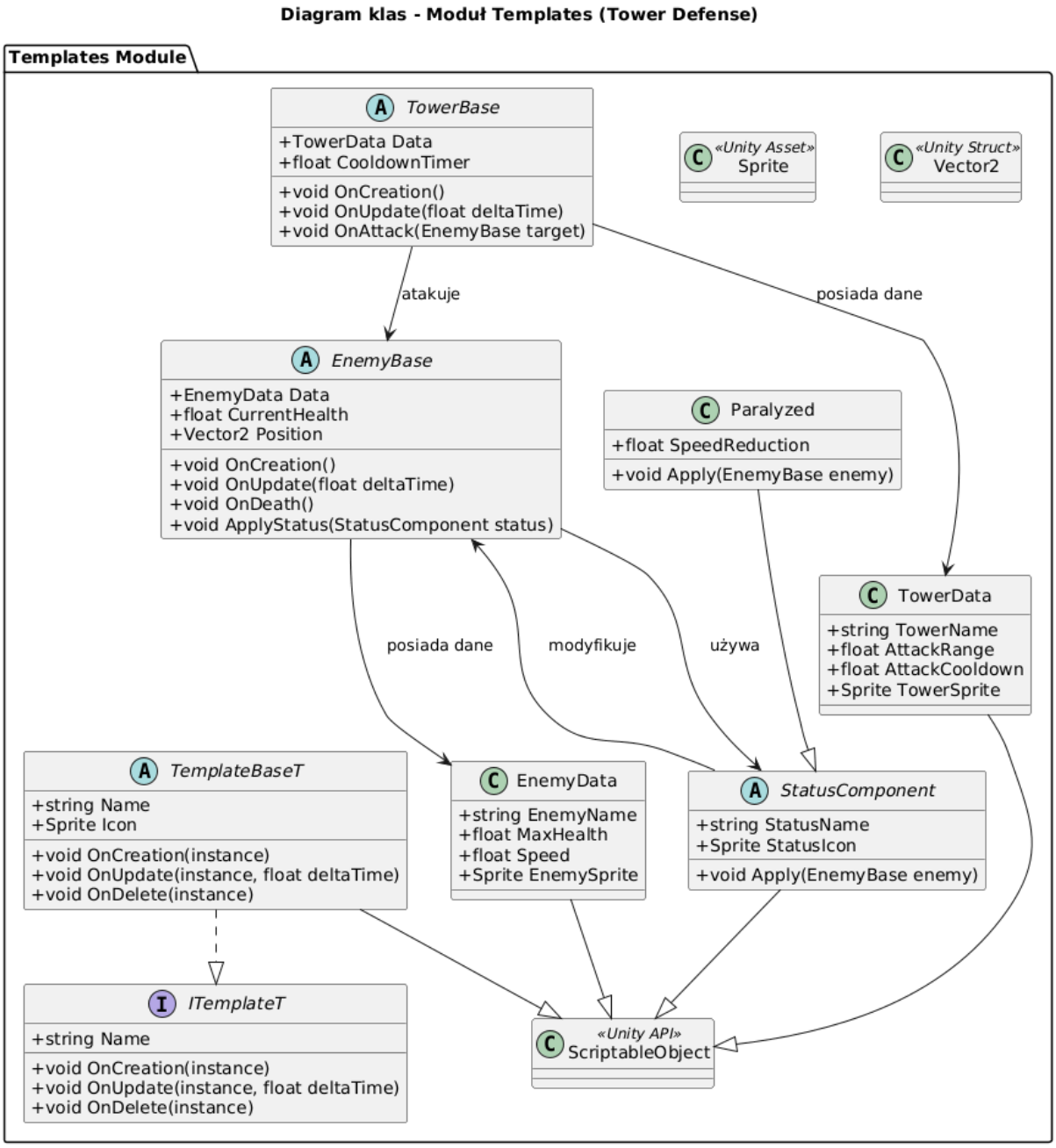
## 8.2 Przykład: Paralyzed

```
[CreateAssetMenu(menuName = "TDCore/Status/Paralyzed")]
public class Paralyzed : StatusComponent
{
    public float SpeedReduction = 3f;

    public override void Apply(EnemyBase enemy)
    {
        enemy.Data.Speed -= SpeedReduction;
    }
}
```

# 9 Integracja z Unity

- Klasy `Data` (np. `EnemyData`, `TowerData`) są `ScriptableObjectami`, które przechowują dane edytowalne w Unity.
- Klasy logiczne (`EnemyBase`, `TowerBase`) są czystymi klasami C#, niezależnymi od silnika.
- Systemy Unity mogą tworzyć instancje logiki i synchronizować je z GameObjectami w scenie.




Rysunek 1: Diagram UML.